II. Design & Architecture

---

1. Architectural Decisions

Web-Based Architecture

A web-based system is chosen over an IDE plugin to provide a centralized Code Quality Dashboard accessible from any browser. This approach simplifies AI model updates without requiring user-side installations.

Data Flow

Information flows through the system as follows:

1. **Ingress**: User pastes code into the JavaScript frontend
2. **Processing**: Python backend performs static and AI analysis
3. **Persistence**: Results and metadata are stored in PostgreSQL
4. **Egress**: Dashboard renders results and dataflow visualizations

---

2. Design Models (With Figures)

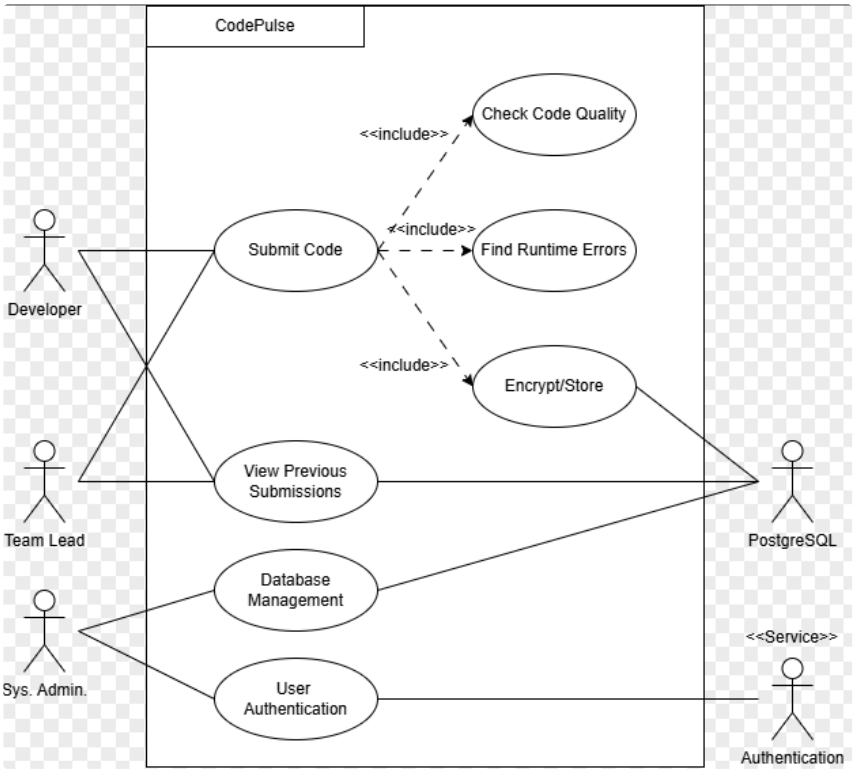
Use Case Diagrams

A visual model to show interactions between the Developer (actor) and the system. These use cases focus on the discrete tasks involving external interaction with the system, where the **Developer** is the primary actor.

- **UC-1: Authenticate User**
  - **Description:** The Developer logs into the web-based dashboard using the authentication system.
  - **Goal:** Ensure secure access to encrypted past submissions and personal metadata.
- **UC-2: Submit Code for Analysis**
  - **Description:** The Developer copies and pastes source code into the JavaScript frontend for processing.
  - **Goal:** Provide the Python backend with the raw data required for static and AI analysis.
- **UC-3: View Code Quality Report**
  - **Description:** The system displays identified bugs, risk warnings, and coding standard deviations (e.g., PEP 8).

- **Goal:** Inform the user of potential errors and structural issues early in development.

- **UC-4: Retrieve Submission History**
  - **Description:** The Developer accesses the PostgreSQL database to view and retrieve past code submissions and their metadata.
  - **Goal:** Allow users to track code improvements or revert to previous analysis states.
- **UC-5: Generate Dataflow Visualization (Reach Goal)**
  - **Description:** The backend converts program logic into an intermediate representation rendered as a flowchart or graph.
  - **Goal:** Help the developer visually understand dependencies and complexity across the codebase.



Some notes for making it match all the use cases after some things changed:

- Ensure your primary actor (the **Developer**) is on the left, and secondary actors (external systems) are on the right.
- **Developer:** Connect to UC-1 (Authenticate), UC-2 (Submit), and UC-3 (View Report).
- **Supabase/PostgreSQL:** Connect as an actor to UC-4 (Retrieve History) and UC-1 (Authenticate) since the system must interact with it to fetch data.
- Add a specific use case inside the boundary for **"Generate Advanced Team Report"** and connect it only to the **Team Lead** actor to reflect their advanced reporting needs.

ER Diagram

A visual model that shows how CodePulse's database is structured with entities and relationships. Because CodePulse is an information system that updates a database based on user requests, its data model follows a **Transaction Processing** pattern.

Primary Entities

- `User`: Stores identity and authentication metadata.
  - *Attributes:* `user_id` (PK), `email`, `password_hash`, `role` (e.g., Developer, Lead), `created_at`.
- `CodeSubmission`: Stores the actual code provided by the user.
  - *Attributes:* `submission_id` (PK), `user_id` (FK), `encrypted_source`, `language`, `timestamp`.
- `AnalysisReport`: The summary result of a specific submission.
  - *Attributes:* `report_id` (PK), `submission_id` (FK), `overall_quality_score`, `ai_risk_level`, `completed_at`.
- `Finding`: Individual issues identified within a report (bugs, standard violations, or refactoring tips).
  - *Attributes:* `finding_id` (PK), `report_id` (FK), `type` (e.g., "bug", "style"), `severity`, `line_number`, `description`, `suggestion`.

Relationships

- **User to CodeSubmission: 1:M** (One user can have many past submissions).
- **CodeSubmission to AnalysisReport: 1:1** (Each unique submission generates one specific report).
- **AnalysisReport to Finding: 1:M** (One report can contain many individual findings or warnings).

Class Diagram

A visual model of a **Repository Architecture** is used to allow for multiple analysis components (static, AI, standards) to interact with a shared internal representation of the code.

- `AnalysisManager` **(Orchestrator):** Manages the overall analysis workflow, calling the parser, AI model, and report generator in sequence.
- `CodeParser`: Converts raw input code into an **Abstract Syntax Tree (AST)** or other internal representation for the analysis tools to "walk" through.

- `BaseAnalyzer`: Performs structural checks, identifying control and data flow patterns without executing the code.
- `StandardsChecker`: A specialized component that compares the code against a chosen `CodingStandard` object (like PEP 8).
- `AIPredictor`: Interfaces with your trained model to analyze the AST for known defect patterns and "suspicious" logic.
- `StaticEngine`: Calculates the complexity of the code, finds dead code, and maps the data flow
- `ReportBuilder`: Aggregates findings from all engines to produce a unified feedback object for the dashboard